

BSc (HONS) COMPUTER SCIENCE

REFLECTIVE REPORT

for

**SMART STUDY: AN AI-BASED COLLABORATIVE LEARNING
AND EXAM PLATFORM**

Student Name	[REDACTED]
Student ID	[REDACTED]
Supervisor Name	[REDACTED]
Unit Name	Undergraduate Project
Unit Code	CIS017-3
Date of Submission	31 th July 2026

Table of Contents

Table of Contents.....	2
1.1 Personal Motivation for Selecting This Project	3
1.2 Initiative and Proactive Approach	3
3. Project Management Reflection and Insights	4
2.1 Planning and Requirements Analysis	4
<i>Tasks.....</i>	<i>4</i>
<i>Challenges and Key Findings</i>	<i>4</i>
<i>Solutions.....</i>	<i>4</i>
2.2 System and Database Design	5
<i>Tasks.....</i>	<i>5</i>
<i>Challenges and Key Findings</i>	<i>5</i>
<i>Solutions.....</i>	<i>5</i>
2.3 Implementation Progress (Front End and Back End)	5
<i>Tasks.....</i>	<i>5</i>
<i>Challenges and Key Findings</i>	<i>6</i>
<i>Solutions.....</i>	<i>6</i>
2.4 Documentation	6
<i>Tasks.....</i>	<i>6</i>
<i>Challenges and Key Findings</i>	<i>7</i>
<i>Solutions.....</i>	<i>7</i>
4. Review of Progress Against the Project Plan	7
3.1 Time Management.....	7
5. Reflection on Work Done and Future Work.....	8
5.1 Challenges Encountered and Solutions (Summary)	8
5.2 Future Work	8
6.Lessons Learned	9
7.Planned Thesis Report Structure	9
8.Conclusion	10
9.Appendix	11

1. Introduction

This reflective report provides a self-evaluation of the development of the Smart Study project, a cross-platform mobile application which supports students in organizing their study material, creating quizzes with the assistance of artificial intelligence (AI), and editing more effectively with the support of spaced repetition and social collaboration. This document aims to review the project in comparison to the original project plan, discuss problems that were faced in the planning, design, and early development of the project, and define the remaining work along with a proposed outline for the final thesis report. This report also encompasses my own project management, self-management, and learning to deliver this report as required in CIS017-3 Undergraduate Project unit.

1.1 Personal Motivation for Selecting This Project

I had an observation that the students here study without any motivation, and when notes and past papers are provided, students do not revisit them in time to retain the information. Current note takers and flashcard apps are either used for storage, for creating quizzes or reminders, and very few have a social and exam-taking aspect where friends can quiz each other. I have worked on smaller mobile products earlier, and wanted to create something that would actually be beneficial to students within my own sphere: An app where a student can organize subjects and topics, upload notes and papers, automatically generate quizzes from the notes based on AI, and be reminded to refine a topic before an exam. The idea was inspired by the way in which students form study groups before an exam, and the competitive feature was done with the students' friends. The exam feature was competitive, and the questions were computer-generated with random numbers to discourage copying – which seemed like the best and most unique aspect of the idea.

1.2 Initiative and Proactive Approach

Before implementation, I conducted informal research by comparing the set of features Smart Study has in mind with other apps in the market like Quizlet, Notion, and Anki, none of which offer AI-generated quizzes, a social exam mode, and adaptive spaced repetition in a single app. It was important to be familiar with both Flutter and Dart, as the front end of the application needed to perform smoothly on both Android and iOS, from the same codebase. To prepare for this, I followed structured tutorials and official documentation to build my understanding of both. I also created wireframes and a clickable prototype in Figma before coding, validating with a small group of my fellow students and making adjustments before coding, when it was easiest to implement. Instead of waiting for formal supervision meetings to come up with questions and concerns, I have maintained a running list of technical decisions and blockers to ensure that supervision discussions are focused and productive, and third-party services (for AI text extraction

and quiz generation) were researched ahead of time to ensure that the feasibility of the AI module is confirmed before it becomes a barrier during the development process.

2. Project Management Reflection and Insights

The following sections discuss each phase of the project so far, in the same order as the planning and requirements analysis, system and database design, and front-end and initial back-end functionality.

2.1 Planning and Requirements Analysis

Tasks

The project started with the review of the project-approved proposal, and then a detailed analysis of the fourteen modules outlined in the project specification (authentication, profiles, subjects, topics, documents, quizzes, AI quiz generation, visibility and permissions, friends, exams, group exams, practice and periodic learning, spaced repetition, notifications, and the performance dashboard). Each module was broken down into user stories and functional requirements, and then those that were critical to a working application were identified through the MoSCoW method, which prioritized the following modules: Authentication, Subject and Topic Management, and Manual Quiz Creation.

Before designing, a requirements traceability matrix and an entity-relationship diagram were created to ensure that for each requirement specified in the system specification (e.g., visibility type, copy permission, and quiz attempt details) there is a corresponding data structure.

Challenges and Key Findings

The AI-based quiz generation module was the hardest to scope out of all the modules, as the requirement was to “extract text from the provided content, contact the AI API, and receive back a set of multiple-choice questions that can be reviewed or edited.” This is a description of the desired behavior, but doesn't specify an exact provider, and I had never embedded an external AI API into a mobile app.

The challenge was also balancing the number of modules with the time available for a semester project; if there were 100 modules to be implemented, the full specification would be closer to a production application than a final year project.

Solutions

I decided to prototype small, stand-alone test scripts using a GPT-based API before deciding to add it as part of the main app, and this proved to be a successful process as it confirmed that the extract-then-generate workflow outlined in the specification was feasible within the project timeline.

For scope, I did agree with my supervisor that the core study and quiz features would come first, followed by the AI generation and spaced repetition features, and then the social/group-exam features would come last, so that a usable core product can be developed even if the stretch features fail to come through.

2.2 System and Database Design

Tasks

For each of the key screens- onboarding, subject and topic browsing, document upload, quiz creation and attempt flows, friends list, and performance dashboard- low-fidelity wireframes and high-fidelity wireframes were created in Figma, and a consistent design system, in terms of colors, typography, and reusable components, was established for the Flutter front end.

Entities for users, subjects, topics, documents, quizzes, questions, quiz attempts, friendships, exams, and notifications were designed using a relational data model, with visibility (private, friends-only, public) and copy-permission fields attached to the entities created as necessary per the specification.

A modular, layered architecture was chosen for the app (presentation, state management, service/repository layer and API client), so that the AI quiz-generation and spaced-repetition logic can be added later without reworking the screens that are already built.

Challenges and Key Findings

It turned out that modelling the visibility and copy permission rules in a consistent way across subjects, topics, quizzes and documents was more challenging than initially thought, as there was no reason to duplicate the access-control logic across the back end of the application for each of these entities.

Choosing a state-management approach for Flutter that would work for simple screens (login, profile) and then scale up to more complex, real-time screens (group exams) needed more research, as it was of great cost to change the approach once it was implemented.

Solutions

I created one Visibility/Permission value object and one shared authorization check that every module uses, meaning that the rule is only stored once and used by multiple modules.

I have done some short proof-of-concept screens with Provider, Riverpod, and BLoC; I chose Provider and a simple repository pattern as it seemed to have a good learning curve and structure for doing this project.

2.3 Implementation Progress (Front End and Back End)

Tasks

The Flutter front end has been developed the most: screens for authentication (register, log in, forgot password), and screens for managing subjects and topics (create, modify, delete, search, and set visibility) have been implemented and can be navigated from end-to-end, without any connection with the back end. The document/note upload interface and screens to create and attempt manual quizzes are also available.

The implementation of the authentication API (registration, login, JWT-based session handling, and password reset), the core CRUD routes for subjects and topics, together with input validation rules (such as email uniqueness enforcement), have been implemented and linked up with the appropriate Flutter screens on the back end.

For the completed modules, a working REST connection has been established between the Flutter app and the back end, allowing the chosen architecture (Flutter front end communicating with a Python (Fast API)-based REST API) to be used for the remaining modules.

Challenges and Key Findings

Initial integration of asynchronous API calls with Flutter's widgets lifecycle caused some bugs related to this, like screens updating without waiting for a network response, mainly in the subject and topic list screens.

It was more complicated than they had anticipated to design the document upload flow to accommodate PDF, JPG, JPEG, and PNG documents and not make the process too lengthy even on a typical mobile connection.

Without manual testing, a small number of mismatches between the data contracts (JSON field names and types) in the Flutter front end and the back-end API were made by hand, which were only discovered during manual testing.

Solutions

Used the Provider pattern with explicit loading, error, and success states for each network screen, eliminating the race condition and providing feedback to the user while data is being loaded.

File uploads were redirected to a dedicated multipart endpoint with server-side size and type checking, and files are streamed to storage instead of being stored entirely in memory, alleviating the performance issue.

A few API response models and a short data-contract document, which both the front-end and back-end code are referring to, have helped greatly to limit integration mismatch since it was put in place.

2.4 Documentation

Tasks

Documents that are needed throughout the project lifecycle, the project proposal, this reflective report, and, subsequently, the contextual report and final thesis are being monitored for their respective deadlines in the same plan as the development tasks.

In addition to the code, a lightweight technical log is being maintained, which will provide a foundation for the writing of the contextual and final report later in the project to build upon, instead of relying on one's memory.

Challenges and Key Findings

Academic report writing, conventions for referencing, and a consistent, formal tone throughout a long document are not as familiar as the technical work, so it took me longer to write this report than it probably would have the technical one.

It can be challenging to maintain up-to-date documentation when implementation changes are happening at a rapid pace – it's easy to put off making notes until a feature is complete.

Solutions

As a guide, I referred to the lecture materials and the provided sample reflective report for this unit in the structure of my report and sought my supervisor's feedback on a draft of my introduction at the beginning of my report, before writing the remainder of the report.

I have changed over to updating the technical log at the end of each work session instead of the end of each feature; this has turned it into a more reliable source for this and future reports.

3. Review of Progress Against the Project Plan

The initial design was to spend the first block of time on requirements analysis and design, then develop the modules iteratively, in priority order, starting with authentication and core content management through to AI quiz generation, spaced repetition, the social/exam features and ultimately testing and evaluation. In contrast to this, the requirements analysis, UI/UX design and data model are finished, and implementation has begun: The Flutter front end for the authentication module, profile module, subject module, topic module and document upload module are all functionally complete, and the back end authentication and subject/topic management APIs are complete and linked. This is about on track, with a little more time spent stabilizing the API integration pattern in Section 2.3, which I anticipate will get to the rest of the modules quicker.

There are two changes to the original plan. First, the position of the AI-based quiz generation module is shifted towards the beginning of the sequence in relation to the friends and group-exam modules, as the feasibility prototyping described in Section 2.1 revealed that there is more technical risk involved and the module is more likely to be handled when there still exists schedule contingency. Second, testing has been broken down from a single stage at the end into a continuous process: testing has been done for every module completed instead of waiting until the end of the project to test all modules.

3.1 Time Management

Time has been monitored for each week, against the various modules in the project plan. Overall development time is about the same as for subjects, topics, and profile, which are well-known, CRUD-style modules, although it has been overestimated for modules that have involved new technology for me, particularly in state-management and file-upload. I'm conscious that the overall schedule is realistic, as

I've re-estimated the remaining modules based on the actual time taken during completion of similar completed modules, rather than the initial more optimistic estimates, and added a contingency buffer for the final testing/ thesis-writing section of the schedule, in case the other modules continue to slip.

4. Reflection on Work Done and Future Work

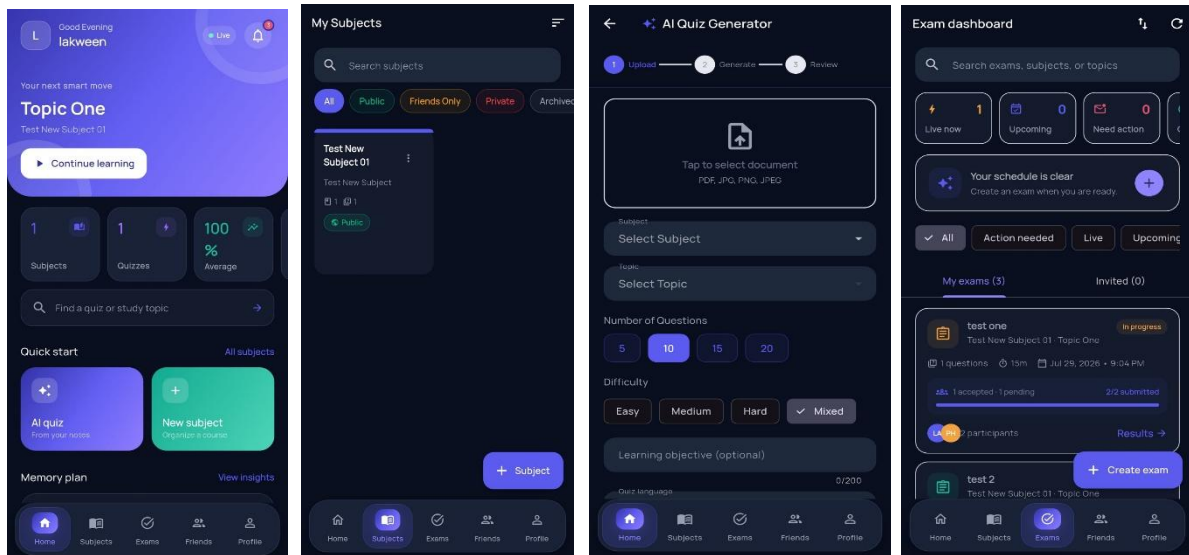


Figure 1: System Interfaces

Overall, the planning, requirements gathering, and system design phases of the Smart Study project are complete, and the front end has been implemented for all of the core study-management screens, while the back end currently supports authentication and subject/topic management. About a third of the total implementation effort has been finished at this stage. The completed work also validated the core architecture—Flutter front end to REST API—which provides confidence that all the remaining modules will be built on the same foundation without needing to make other changes.

4.1 Challenges Encountered and Solutions (Summary)

The biggest hurdles that have arisen have been the clean integration of asynchronous back-end calls into the Flutter widget lifecycle; ensuring consistency between the front-end and back-end data contracts, which were developed concurrently; and applying the AI feature of generating quizzes to a realistic scope of the project. All of these have been solved as follows: a consistent pattern of state management, a common data-contract reference, and early feasibility prototyping of the integration of AI.

4.2 Future Work

In the coming weeks, the primary focus will be on:

- Writing the remaining back-end API endpoints for document/note upload, Quiz management and Quiz attempts, and linking them to the existing front-end screens.

- Using the AI-based quiz generation module that involves extracting text from uploaded PDFs/images, connecting with a GPT-based API, and a user review stage before saving a generated quiz.
- Constructing spaced-repetition scheduling logic and a notification module to remind users to edit according to their quiz results.
- Making the friends module and individual and group exams features work. The friends module is implemented along with the features of individual and group exams, which include randomized and shuffled sets of questions for group exams.
- Creating a performance dashboard (score trend, topic-wise/substantive-wise charts).
- Performing functional and usability tests in structured mode for all modules and resolving the defects found in the tests.
- Maintaining the technical log throughout development and writing the contextual report and then the final thesis.

5. Lessons Learned

The learnings I hope to take with this stage of the project, as well as the project management, will be useful in the remaining work and for my future software career. In a technical sense, I have developed my knowledge of Flutter's widget lifecycle and state management, as well as defining clear, well-documented contracts for REST APIs, and the practical compromises needed when creating a role- and permission-based data model. In particular, the visibility and copy permission requirements helped me grasp the authorization design, which is easy to informally describe, but can be difficult to implement uniformly.

In terms of project management, it is useful to have the module list prioritized with MoSCoW and to review and update the time estimates based on experience and not first guesses – this makes the plan more credible and less likely to be used as a cover for slippage. Having a running technical log has also been found to be beneficial not just for documentation. It has also made the supervision meetings more focused, as issues and decisions are written down as they occur, rather than told again in retrospect. Last, but not least, I have prototyped the feature with the highest risk (AI quiz generation) earlier in the project, not waiting until other modules are complete to find out we have an unpleasant surprise later on, which I also plan to do with the other group-exam and spaced-repetition modules.

In terms of self-management, it's improved during this stage: I've gotten more comfortable raising blockers and am more disciplined in keeping documentation up to date with code, and more realistic on scope, which I expect to add more to as the project continues in its more collaborative and AI-driven modules.

6. Planned Thesis Report Structure

This document is meant to be used as a summary for the final thesis report. The table below is an outline of the chapters and a brief description of what is planned to be included in each chapter, to satisfy the requirements to outline the report and describe each chapter.

Section	Planned Content
Cover Page	Project title, student name and ID, submission date, module name and assignment reference.
Abstract	A concise overview of the Smart Study project, its aim and its contribution.
Acknowledgements	Recognition of the supervisor, participants and resources that supported the project.
Table of Contents	Full listing of chapters, sections and figures.
Chapter 1: Introduction	1.1 Project Background; 1.2 Aim and Objectives; 1.3 Project Scope; 1.4 Structure of the Report.
Chapter 2: Literature Review	Review of existing study, flashcard, and quiz applications; spaced-repetition research; and AI-based question-generation techniques, positioning Smart Study against this background.
Chapter 3: Methodology and Design	Development methodology (Agile/Scrum), requirements analysis, system architecture, data model, and UI/UX design decisions.
Chapter 4: Development and Implementation	Description of the Flutter front-end and back-end implementation for each module, including the AI quiz-generation pipeline and spaced-repetition logic.
Chapter 5: Testing and Results	Test plan, test cases, and results for functional, usability, and AI-accuracy testing, with supporting screenshots.
Chapter 6: Evaluation	Evaluation of the finished application against the original objectives, including user feedback.
Chapter 7: Conclusion and Future Work	Summary of achievements, project limitations, and recommendations for future development.
References	Full Harvard-style reference list of all sources cited.
Appendices	Supporting material such as the Gantt chart, ethics approval, and additional screenshots or code listings.

7. Conclusion

This report has examined the development of the Smart Study project in relation to the plan, discussed the issues that arose while planning, designing, and implementing the project and the way these issues were overcome, and identified the tasks that are yet to be completed before writing the thesis. The front end of the application is almost all the way ready with the main screens for study management, and the back end currently includes authentication and subject/topic management; a module that creates AI quizzes with spaced repetition is next in the order of development. The front end of the application is nearly fully functional in terms of its core screens for study management, while the back end already supports

authentication and subject/topic management, and development of the module for creating AI quizzes with spaced repetition is next in line. The project continues to stay on schedule, and the changes that have been implemented, such as performing AI feasibility work earlier in the project and testing throughout the project, are designed to do this. The lessons learned to date, including just contract discipline at the API level and proactive self-management, will be translated into the remainder of the project, which will be more complex.

8. Appendix

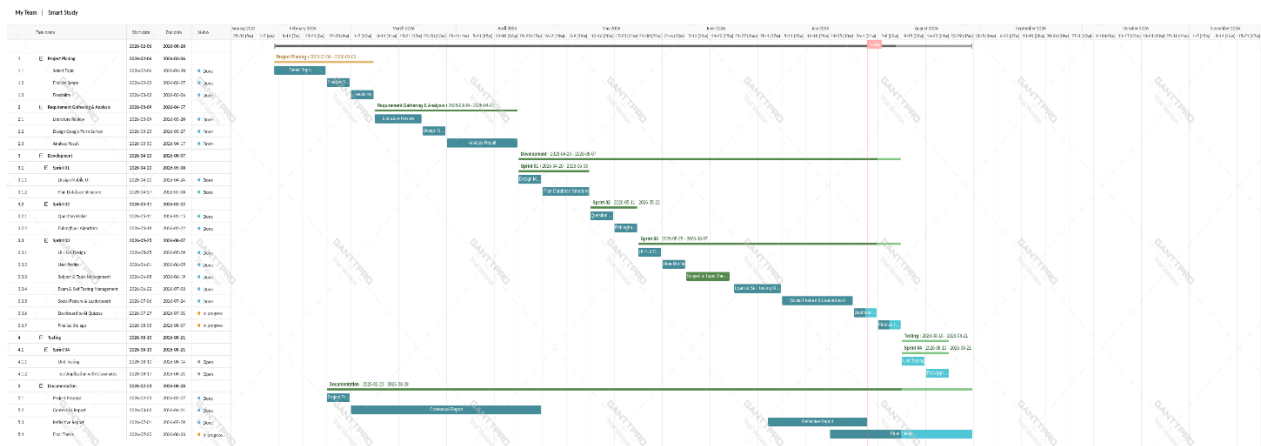


Figure 2: Revised Gantt Chart